

Lecture 3

Search Algorithms 2

Informed search

Intelligent Systems
Vrije Universiteit

1

Today we will be looking at a different kind of search algorithm than last week, namely informed search. First, let's recap what we've seen already.

Overview

- 1 Recap
- 2 Uninformed Search Continued
- 3 Search Directions
- 4 Informed Search & Heuristics
- 5 (Greedy) Best-First Search
- 6 Local Search & Hill-Climbing
- 7 A & A* Search

Part 1: Recap

Intelligent Systems
Vrije Universiteit

Recap Lecture 2

Uninformed search uses only information available in the problem definition.

Breadth-first search (BFS) explores the shallowest node first, moving left to right first

Depth-first search (DFS) explores the deepest node first, moving top to bottom first

Performance Metrics

Completeness	does the strategy always finds a solution if one exists?
Optimality	does it find the least-cost solution?
Time complexity	what number of nodes are generated/expanded?
Space complexity	what number of nodes are stored in memory during search?

4

Uninformed search algorithms allow us to find a goal state in a state space by going through every node in a search tree. Breadth-first search does this by moving in a left-to-right manner and exhausting every node at a certain level before moving down. Depth-first search moves top-to-bottom and exhausts every node in a single path before backtracking and exploring nodes at the levels it has already crossed.

Notice how these algorithms follow the same pattern every time, regardless of the problem.

There are four performance measures that we can use to compare how well a search algorithm works. **Complete** means that it finds a solution when one exists, **optimal** means that the found solution is the best one, **time complexity** is how many nodes are expanded before the goal node is found, and **space complexity** is how many nodes are stored in the fringe or memory.

Performance of Search Strategies

Performance Measure	Breadth-First	Depth-First	Depth Limited	Iterative Deepening
Completeness	YES*	NO	YES, if $l > d$	YES
Time	b^d	b^m	b^l	b^d
Space	b^d	bm	bl	bd
Optimality	YES*	NO	NO	YES

*BFS finds the best solution when the branching factor is finite.

b the maximum branching factor of the search tree

d the depth of the least-cost solution

m the maximum depth of the state space problem (may be ∞)

As compared to the algorithms that we're now familiar with, iterative deepening performs better than depth-limited search because it is *optimal*. This means that there is a guarantee that the solution found is the one with the cheapest cost from initial state. The advantage of iterative deepening over breadth-first search, which is also optimal, is that iterative deepening uses significantly less memory in the process. How? Because iterative deepening throws away its entire memory after every unsuccessful iteration and systematically repeats it with a depth of +1.

Part 2: Search Directions

Intelligent Systems
Vrije Universiteit

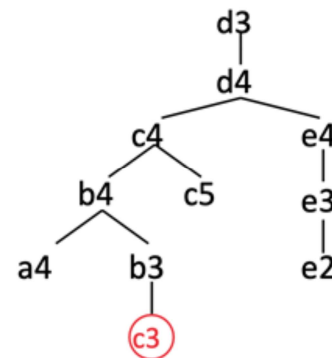
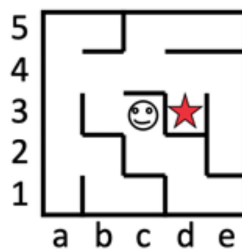
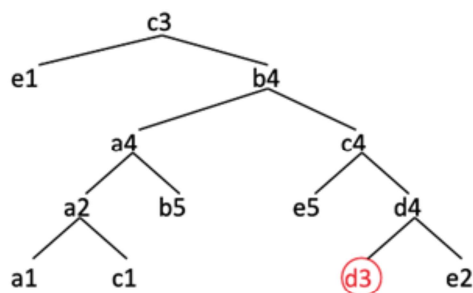
6

Thus far we have been focused on reaching a goal state from some node in a tree; however, this is not always the smartest approach. In some situations, the information we have about the problem's goal and branching factor is an indicator that we should start with the goal and move backwards.

Search Directions

Data-driven starts at the current state.

Goal-driven starts at the goal state.



Search direction is a relative concept that depends on how you represent the state-space.

7

In the state-space search problems we have discussed so far, we took a **data-driven** approach, which means we start from the current state and reason backward to the goal state. Another possibility is to take a **goal-driven** approach in which we start at the goal state and reason our way forward to the goal state. In this maze scenario, a data-driven tree starts from the current position of the agent at c3 and try to reach d3. Alternatively, we can represent that state-space such that the initial state d3 is the goal itself, and we reason towards the current state of c3.

Choosing a Search Direction

1. Is there a clear, distinct goal?
2. Which search direction results in a bigger branching factor?

Example Is Mark Rutte a descendant of Willem van Oranje?

Data-driven approach

Check children, grandchildren, great grandchildren, etc. of Willem van Oranje:

For n generations, 3 children per n : 3^n

$3^{10} = 59049$ branching factor

Goal-driven approach

Check parents, grandparents, great grandparents, etc. of Mark Rutte:

For n generations, 2 parents per n : 2^n

$2^{10} = 1024$ branching factor

There are two useful questions for deciding whether a data- or goal-driven approach is appropriate for the problem, (1) is there a clear, distinct goal and (2) which direction has a bigger branching factor. For example, given the problem of finding whether Mark Rutte is a descendant of Willem van Oranje, it is useful to visualise a family tree. A data-driven approach would have a branching factor of 3^n because we can imagine that there is a historical average of three children per set of parents since the life of Willem van Oranje. A goal-driven approach is more logical because it has a branching factor of only 2^n , because there are two parents for every child.

Part 3: Informed Search & Heuristics

Intelligent Systems
Vrije Universiteit

9

So far we have looked at uninformed search methods, which can only use the problem information to generate a search tree. Although not necessarily computationally expensive, these methods aren't always optimal or complete. To solve these problems we have the notion of informed search, which are algorithms that can search for a goal state a bit more intelligently.

Simple Tree Search → Informed Search

```
function TREE-SEARCH(problem, fringe) return a solution or failure
  fringe INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if EMPTY?(fringe) then return failure
    node ← REMOVE-FIRST(fringe)
    if GOAL-TEST[problem] applied to STATE[node] succeeds
      then return SOLUTION(node)
    fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
    fringe ← INSERT-ORDERED(EXPAND(node, problem), fringe)
```

Informed search uses an intelligent method to decide which nodes to expand next called a **heuristic**.

Uninformed fringe

Informed fringe

10

In simple tree search pseudocode, we ask the program to insert all nodes in the tree into the fringe, which is the data structure that keeps these nodes. In **informed search**, the nodes are inserted in a particular order. This order is managed by the particular algorithm and the **heuristic** it uses.

Heuristics

A **heuristic** $h(n)$ determines the evaluation function $f(n)$ that an informed search algorithm uses. It is

“A rule of thumb, simplification, or educated guess that reduces or limits the search for solutions in domains that are difficult and poorly understood.”

$h(n)$ is calculated for every node in the search tree; it's the estimated cost of the cheapest path from the current node n to the goal node g .

→ if $n = g$, then $h(n) = 0$ because we are at the goal itself

11

Think about the thought process that you use when playing a game of chess. Rather than think through every possible move that you could make, it is smarter to only think through the moves that you know will give you an advantage over your partner. This type of thought process is a **heuristic**, or a rule that you use to evaluate the possible moves that you can make.

We use heuristics to make decisions based on our partial knowledge, because complete information is often not available. Just like your performance on a game of chess, heuristics used by informed search algorithms are imperfect and can lead them down the wrong path (literally).

Best-First Search

Best-first search is an informed search method that expands the node with the most promising outcome.

Method: expand to a node based on an **evaluation function $f(n)$** , which measures the distance from the node to the goal.

Implementation: fringe is a queue sorted in decreasing order of desirability
lower distance to the goal = higher desirability

Special cases: Greedy, Hill Climbing, A, and A* search.

12

Best-first search is the most basic form of informed search. This algorithm explores a tree by expanding the nodes that seem the most promising for finding the goal node. But how does it decide which node is the most promising? This is done by an evaluation function, which orders the nodes depending on how desirable they are.

In best-first search, the evaluation function is simply the heuristic $h(n) = f(n)$.

Heuristic Functions *Examples*

Goal is to order the tiles clockwise, starting with 1 in the top left corner.

Heuristic 1

No. of incorrectly placed tiles

$$h\left(\begin{array}{|c|c|c|}\hline 2 & 8 & 3 \\ \hline 1 & 6 & 4 \\ \hline & 7 & 5 \\ \hline\end{array}\right) = 5$$

Heuristic 2

Sum of distance of tiles from the goal position
(**Manhattan distance**)

$$h\left(\begin{array}{|c|c|c|}\hline 2 & 8 & 3 \\ \hline 1 & 6 & 4 \\ \hline & 7 & 5 \\ \hline\end{array}\right) = 6$$

Heuristic 3

2x the number of direct tile swaps

$$h\left(\begin{array}{|c|c|c|}\hline 2 & 8 & 3 \\ \hline 1 & 6 & 4 \\ \hline & 7 & 5 \\ \hline\end{array}\right) = 0$$

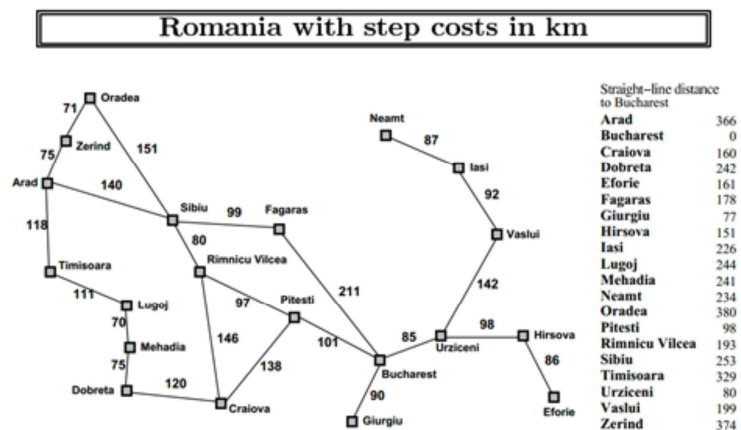
13

Imagine a jigsaw puzzle in which we want to order the tiles in order of their number, starting with 1 in the top left corner. We can think of several different heuristic functions which measure how *good* a node is compared to its neighbours. One common type of heuristic function is **Manhattan distance**, which is the distance between two points along axes at right angles. Imagine moving from one intersection to another in Manhattan; the only option is to move at right angles around the city blocks.

Heuristic Functions *Examples*

Goal is to travel from Arad to Bucharest through other Romanian cities.

The **straight-line-distance (SLD)** heuristic is the cost of a straight line towards the goal.
e.g. how a bird would fly

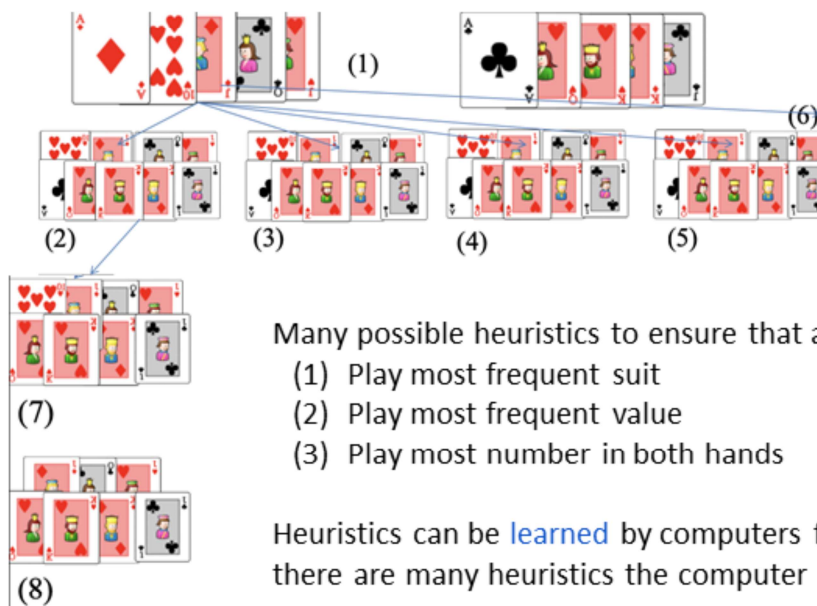


14

Another important heuristic function is **straight-line-distance (SLD)**, which is the distance between two points if we were to draw a straight line from one to the other. Going back to the city example, straight-line-distance would be the distance we would have to travel from one intersection to the next if we were a bird flying over all the buildings and possible obstacles.

In the given map of Romanian cities, we can observe the costs of the paths between cities, but now have the straight-line-distances in km between each node and the goal, Bucharest.

Heuristic Functions *Schnaptience*



Recall [Schnaptience](#), a single-player game where you must alternate playing a card from each deck which either (1) matches suit or (2) matches value.

Many possible heuristics to ensure that all cards can be played eventually:

- (1) Play most frequent suit
- (2) Play most frequent value
- (3) Play most number in both hands

Heuristics can be [learned](#) by computers for games. For example in chess, there are many heuristics the computer has learned and can alternate using.

Heuristics are usually very task-specific, but they all share a common goal; finding the best path to reach the goal state. In the case of Schnaptience, the goal is to play all cards in both hands of the player such that each subsequent card either matches suit or value or the previous one. Heuristics like (1) playing the most frequent suit or (2) playing the most frequent value that the player is holding is a more intelligent method of choosing which card to play next. But there are also heuristics that could be implemented that do not increase performance of the player, such as always playing the lowest value cards.

These kinds of heuristics are what motivate the actions of NCPs in computer games such as chess.

Properties of Heuristics

Informedness for two heuristics h_1 and h_2 , if $h_1(n) \leq h_2(n)$ for all states n in the search space, then h_2 is said to be more informed than h_1 .

- Advantage: Fewer nodes need to be searched with h_2
- Disadvantage: h_2 is often more expensive to calculate

Admissibility the heuristic finds the shortest path to the goal if one exists.

Monotonicity the heuristic is monotone if when a goal state is found, there is a guarantee that the goal state won't be found elsewhere at a lower cost.

16

Let's look at three properties of heuristics that help us compare them with each other. A heuristic function h_2 is said to be more **informed** than h_1 when $h_1(n) \leq h_2(n)$. In other words, given the same outside knowledge about the problem, it is guaranteed that h_1 will not find the optimal solution while doing less work.

A heuristic is **admissible** if it never overestimates the cost it will take to reach the goal; it will find the least cost solution if one exists. **Monotonicity** means that the actual cost is never less than the heuristic cost; the algorithm is *locally admissible*. If a vertex is rediscovered, the path will not be shorter.

Part 4: (Greedy) Best-First Search

Intelligent Systems
Vrije Universiteit

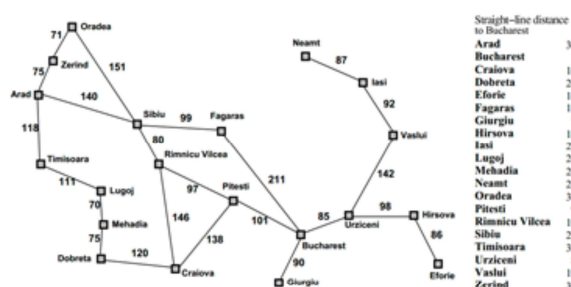
17

Let's look further into best-first search, namely a variant called greedy best-first search.

Greedy Best-First Search

Greedy best-first search expands the node with minimal heuristic function $h(n)$.

- Algorithm **greedily** chooses a new successor and can not backtrack and revise a decision as with best-first search
- The evaluation function $f(n) = \text{the heuristic } h(n)$



The difference between **greedy** and regular best-first search is an inability to backtrack and revise any node expansions that have already been made. As the name suggests, this algorithm greedily chooses nodes for expansion based on which one has the minimum heuristic function.

Applied to the map of Romania, this algorithm would choose a successor city solely on its straight-line-distance to Bucharest given in the list, disregarding the cost of the path to get there.

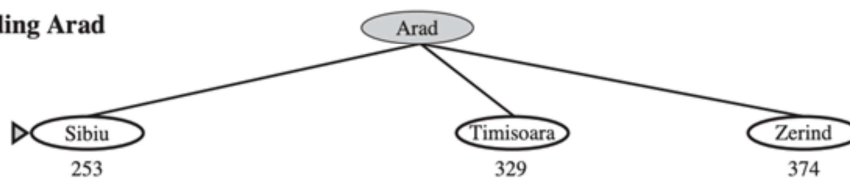
Greedy Best-First Search Example

Goal solve the Arad to Bucharest problem with Greedy Best-First Search.

(a) The initial state



(b) After expanding Arad



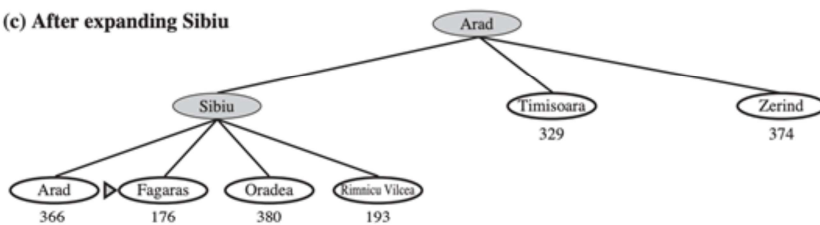
Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

The number under each city represents its respective SLD to Bucharest, shown in the list on the right. Starting from the initial state, $h(\text{Arad})$ is 366. Our greedy algorithm now has three possible cities to expand to, whose heuristic functions are $h(\text{Sibiu}) = 253$, $h(\text{Timisoara}) = 329$, and $h(\text{Zerind}) = 374$. Greedy will choose to expand Sibiu because it has the lowest heuristic value of all three.

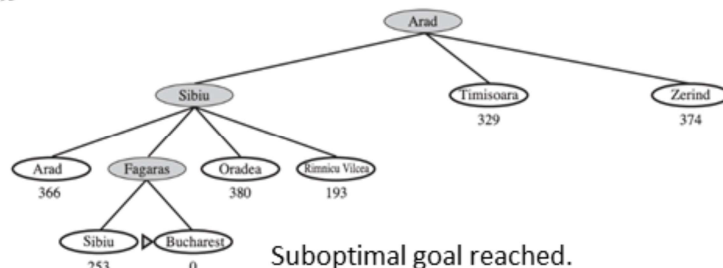
Greedy Best-First Search Example

Goal solve the Arad to Bucharest problem with Greedy Best-First Search.

(c) After expanding Sibiu



(d) After expanding Fagaras



Suboptimal goal reached.

Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

This process continues for every node that is expanded. After we expand to Sibiu, greedy will compare the SLD for not only Fagaras, Oradea, and Rimnicu Vilcea, but also for where we just came from, Arad, and its children Timisoara and Zerind. The final path taken by greedy best-first search is, Arad → Sibiu → Fagaras → Bucharest.

Observe that after every expansion, the nodes available for expansion include the node that we just came from, which can lead to loops. For example, if the SLD of Arad was 100, then the algorithm would have gotten stuck in the following loop,

Arad → Sibiu → Arad → Sibiu ...

Greedy Search Evaluation

Greedy search is **incomplete** even in a finite search space.

- Repeats nodes that have already been expanded, ending up in cycles

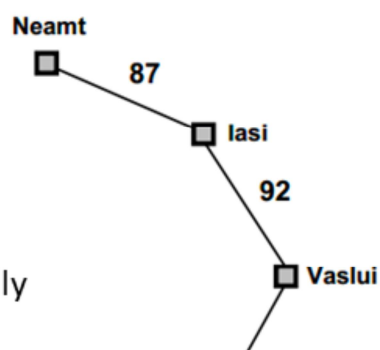
Not optimal because it can lead to false starts.

e.g. from Iasi to Neamt = 87 (dead end)

from Iasi to Vaslui = 92 (correct path)

Time complexity is b^m ; all nodes kept in memory

Space complexity is b^m ; good heuristic dramatically improves this



Because of the greedy algorithm's vulnerability to loops, it is an incomplete search strategy that may never find the solution if one exists. It is also not optimal because it may lead to false starts, which would be the case if the initial goal was Iasi. Instead of moving to Vaslui, which is the necessary successor node to exit this dead end, it would go towards Neamt.

The time complexity of the greedy algorithm increases exponentially with the depth of the tree. Its space complexity is also troublesome because greedy best-first search keeps all possible nodes in its memory at the same time.

Performance of Search Strategies

Performance Measure	Breadth-First	Depth-First	Greedy Best-First
Completeness	YES*	NO	NO
Time	b^d	b^m	b^m
Space	b^d	bm	b^m
Optimality	YES*	NO	NO

*BFS finds the best solution when the branching factor is finite.

b the maximum branching factor of the search tree

d the depth of the least-cost solution

m the maximum depth of the state space problem (may be ∞)

When compared to breadth-first and depth-first search, it becomes clear that greedy best-first search may not be a better search method to use. This is highly dependent on the type of state space we are confronted with and which heuristic is used in greedy best-first.

Part 5: Local Search & Hill Climbing

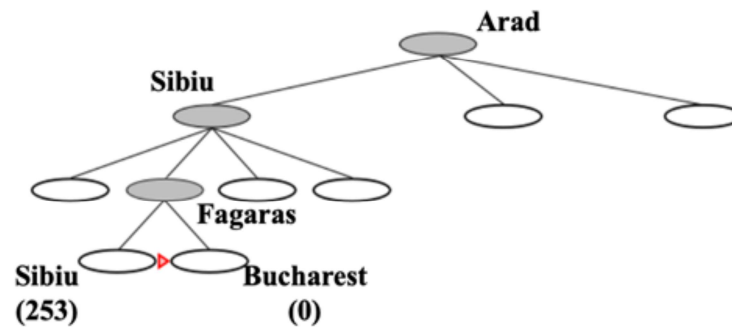
Intelligent Systems
Vrije Universiteit

23

One of the problems present in all the algorithms we've covered so far is their inefficiency with respect to memory. In this section we'll look at search algorithms that restrict their search to increase efficiency.

Global vs. Beam (Local)

What we've covered so far is **global** Greedy Search, which keeps all open nodes in the fringe.



We can increase efficiency by restricting the size of the fringe creating **Beam Search**, which is **local**.

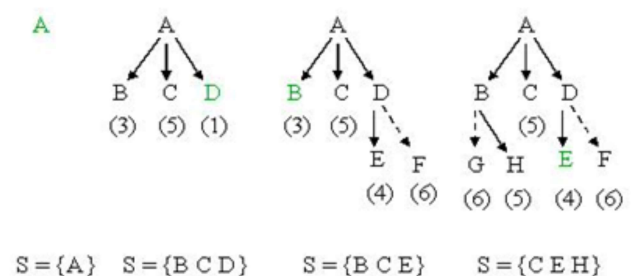
In Greedy search, the fringe inefficiently stores many of the nodes in its memory. Now we explore another option, which is to restrict the fringe to a certain length, called **beam search**. In the Romanian city example, this could mean evaluating two out of all possible successor cities before choosing one. By only looking at a *beam* of the possible nodes, the algorithm may miss a solution, or even the optimal solution, but it makes this sacrifice to increase its efficiency.

Local Beam Search

Local Beam Search expands to the most promising nodes, only expanding the k best nodes at each level.

Method: at each state, choose most promising state according to a heuristic.

Implementation: keep k states;
use **breadth-first search** to traverse each level's k best nodes of all successors.



Beam Search $k = 3$

25

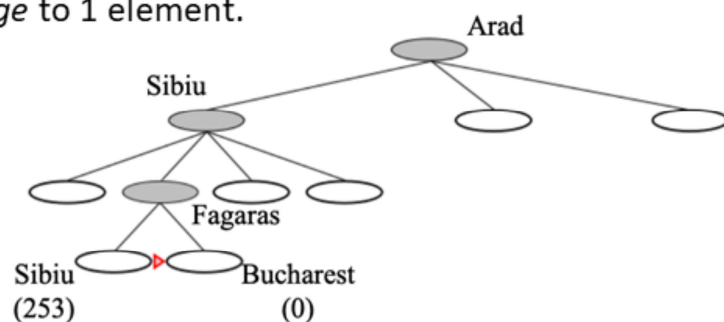
Local beam search is a heuristic search algorithm that evaluates a graph by expanding the most promising node in a restricted fringe. It progresses level-by-level in a BFS fashion and moves downwards by evaluating only the k best nodes. All other nodes are not taken into account. The width of the *beam* limits the memory required to complete the search.

Hill Climbing (Beam search with $k=1$)

Hill Climbing compares neighbouring nodes and selects the one which optimizes the current cost.

Method: at each state, choose most promising state according to a heuristic.

Implementation: limit the *fringe* to 1 element.

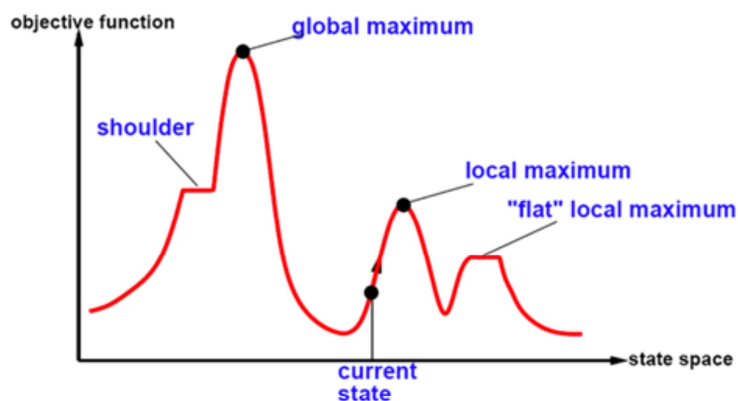


26

We can take this concept to the extreme and decide to restrict the fringe to the single best node, in an algorithm called **hill climbing**. Hill climbing is also a form of local search and Greedy best-first search because the current node it is evaluating is compared to the immediate surroundings to evaluate whether it can move closer to, or find, the solution. At each node, the current best solution is compared to the direct neighbours of the current node according to the heuristic being used.

State Space Landscape

The **state space landscape** is a useful graphical representation of an algorithm's current state x and the objective (cost) function y .



27

Informed search algorithms use mathematical functions to optimise the distance from the current node to the goal. We can represent every node that is explored in a search tree in terms of the value of its objective (heuristic) function. The **state space landscape** is a useful representation to visualise the most promising node at every moment in the search. The goal of the algorithm is to find the global maximum, which represents the goal state.

Hill climbing derives its name from its behaviour in the state space landscape.

If we are using hill climbing and are at the node 'current state', the algorithm will only choose the node that allows it to move up, in this case to the right.

Hill climbing will only move to a point with higher heuristic value, and terminates otherwise. If this algorithm continues to move upwards we will reach a local maximum, at which point it gets stuck.

A possible solution to this issue is to *backtrack*, like in DFS, and explore the second most optimal path from the current node. However, hill climbing erases everything but the most promising node from its memory, so it is unaware of the second best or any other alternative and can not perform backtracking.

Local Beam Search Evaluation

Choosing top k states of all successors is not the same as k states run in parallel.

→ Searches that find good states recruit other searches to join them

Incomplete because all k states often end up on same local hill.

Stochastic beam search: choose k successors randomly, biased towards good ones.

→ Observe the close analogy to natural selection.

28

Notably, local beam search is susceptible to getting stuck in a dead-end on a locally maximal hill, as demonstrated using the state space landscape. A powerful aspect of beam search is that all elements in the beam can be evaluated at the same time, meaning its *parallel feasible*. However, we may not find all solutions, let alone the most optimal one. This algorithm is still highly dependent on the type of heuristic that is used to evaluate each node. Given a bad heuristic and unfortunate distribution of nodes, the time complexity of this algorithm is also not very good.

Since local beam search always chooses the k top successors, these states may be similar and get us stuck in a small local space. **Stochastic beam search** combats this issue by choosing k states with a random factor, biased towards desirable ones.

Part 6: A & A* Search

Intelligent Systems
Vrije Universiteit

29

A and A* (pronounced A star) search are some of the most popular search algorithms in AI. The greatest advantage of using A search over local beam search is that it solves our problem of optimality.

A Search

A **search** is the most well known form of best-first search which evaluates nodes by combining two costs

$g(n)$, the cost to reach the node from initial node

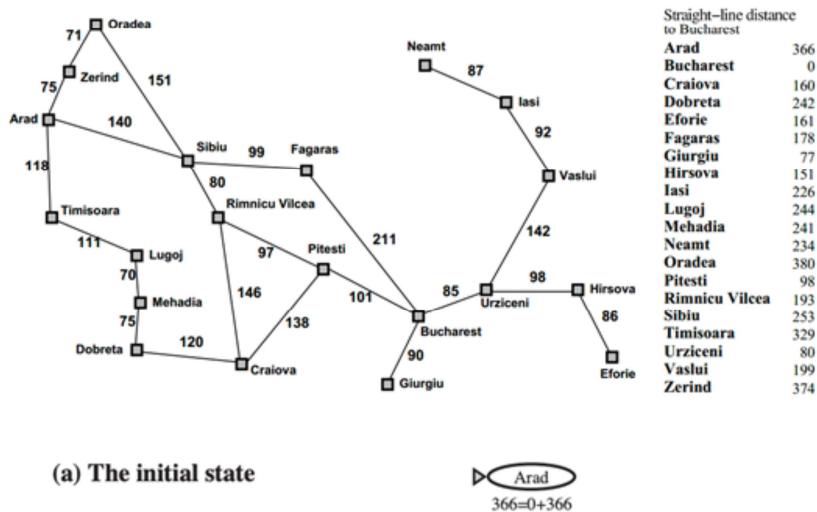
$h(n)$, the cost from node to goal from current node (heuristic)

$$f(n) = g(n) + h(n)$$

where $f(n)$, estimated total cost of path through n to the goal.

In addition to the heuristic value of the successor node $h(n)$, **A star** evaluates the cost that has already been made to reach that node from the initial node $g(n)$. These two functions are summed to result in the evaluation function $f(n)$, which represents the costs made so far and the estimate for what is still needed to solve the problem.

A Search Example



$$f(n) = g(n) + h(n)$$

Arad

$g(n) = 0$, the cost so far is zero.

$h(n) = 366$, SLD to Bucharest in km.

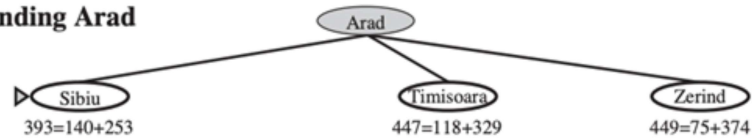
$f(n) = 0 + 366$, estimated cost of path to goal.

The function $f(n)$ is calculated for every node along the search path, including the initial state Arad. $g(n)$, or the cost so far, is zero because we have not traveled yet. $f(n)$, or the distance to the goal, is 366, derived from the straight-line-distance to Bucharest on the right.

Thus, $f(n) = 0 + 366 = 366$.

A Search Example

(b) After expanding Arad



$f(n) = g(n)$ cost of path + $h(n)$ SLD to Bucharest

Arad → Sibiu

$$g(n) = 0^* + 140 = 140$$

$$h(n) = 253$$

$$f(n) = 140 + 253 = 393$$

Arad → Timișoara

$$g(n) = 0^* + 118 = 118$$

$$h(n) = 329$$

$$f(n) = 118 + 329 = 447$$

Arad → Zerind

$$g(n) = 0^* + 75 = 75$$

$$h(n) = 374$$

$$f(n) = 75 + 374 = 449$$

$0^* = g(n)$ for Arad

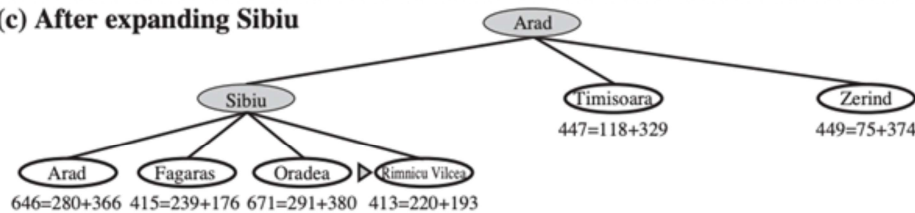
Best choice is Sibiu → 32

We calculate these values for each possible successor of Arad. Starting with Sibiu, $g(n)$ is 140 because the cost of the path from Arad to Sibiu written on the edge between the two cities is 140. We add the value of $g(n)$ for Arad, which was zero. $h(n)$ can be read from the straight-line-distance to Arad as 253, resulting in $140 + 253 = 393$.

This calculation is repeated for Timisoara and Zerind. The final evaluation values are compared with one another, and Sibiu is chosen because it has the lowest cost.

A Search Example

(c) After expanding Sibiu



$f(n) = g(n)$ cost of path + $h(n)$ SLD to Bucharest

Sibiu → Arad	Sibiu → Fagaras	Sibiu → Oradea	Sibiu → Rimnicu Vilcea
$g(n) = 140 + 140 = 280$	$g(n) = 140 + 99 = 239$	$g(n) = 140 + 151 = 291$	$g(n) = 140 + 80 = 220$
$h(n) = 366$	$h(n) = 179$	$h(n) = 380$	$h(n) = 193$
$f(n) = 280 + 366 = 646$	$f(n) = 239 + 179 = 415$	$f(n) = 291 + 380 = 671$	$f(n) = 220 + 193 = 413$

Best choice is Rimnicu Vilcea → 33

Once Sibiu is expanded, A star compares evaluation functions for each successor city again. Notice that Arad is also included in possible successor cities. This is because A star does not keep in its memory whether a node has already been expanded, thus it has forgotten that we have been to Arad. This means that, if Arad has the lowest evaluation function, we will simply go back.

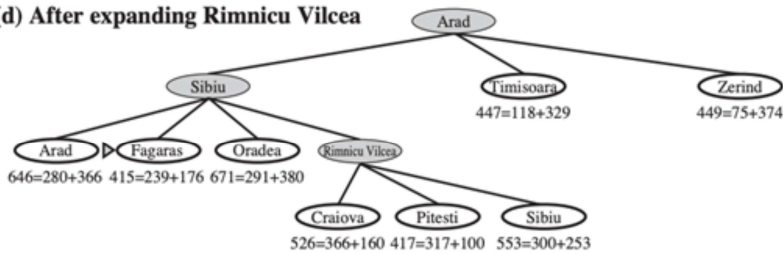
Secondly, it is important to observe that $g(n)$, the cost so far, is cumulative.

Recall that the cost from Arad to Sibiu was 140, and notice that every $g(n)$ calculation adds 140 to the cost of the path from Sibiu to the next city.

Of all possible successor cities, Rimnicu Vilcea has the lowest evaluation function and is chosen for expansion.

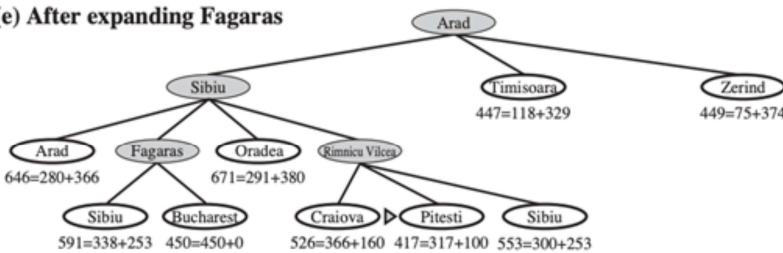
A Search Example

(d) After expanding Rimnicu Vilcea



Observe that the cost $g(n)$ is **cumulative**, meaning that every new path cost is added to the existing node to reach the node.

(e) After expanding Fagaras

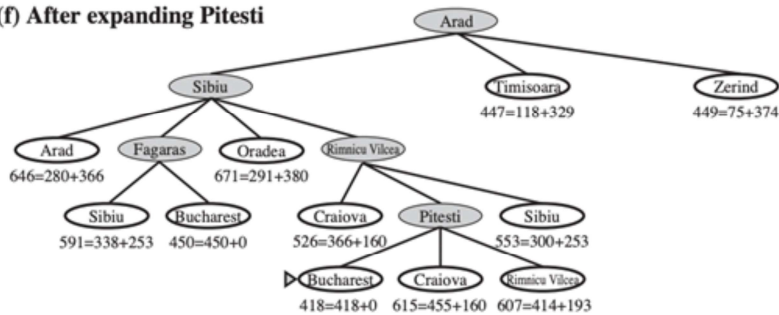


A star will **backtrack** to previous nodes if their children have lower evaluation functions than the children of the current node.

Not only are the children of the most recently expanded node evaluated, but the other cities that were previously not chosen for expansion are still in the competition. For example, Fagaras is initially expanded into Sibiu and Bucharest (the goal state!). Rather than expand to the goal state immediately, A star *backtracks* and chooses Pitesti for expansion because it has a lower evaluation function.

A Search Example

(f) After expanding Pitesti



The final path for A Search:

Arad →

Sibiu →

Rimnicu Vilcea →

Fagaras →

Pitesti →

Bucharest .

This example of A star search demonstrates the cumulative nature of $g(n)$, the possibility to backtrack, and the possibility to expand to a node that was already expanded.

A* Search

A* search is A search with an **admissible** heuristic, meaning it **never overestimates** the cost to reach the goal.

Admissible heuristics are **optimistic**;

$$h(n) \leq h^*(n) \quad \text{where } h^*(n) \text{ is the true cost from } n$$

$$h(n) \geq 0 \quad \text{meaning } h(G) = 0 \text{ for any goal } G$$

e.g. $h_{\text{SLD}}(n)$ can never overestimate the actual road distance.

The only difference between A and A* search is the heuristic function that is used. A* search requires that the heuristics is **admissible**, meaning it never overestimates the cost to reach the goal. For example, the straight-line-distance is admissible because the path eventually found can only be equal in length or longer, since SLD is the shortest possible distance between two lines. Written formally, h^* is the true cost from n and h is the estimated cost. For a heuristic to be admissible, and thus optimistic regarding the cost, the estimated cost must always be less than the true cost.

Admissible Heuristics Example

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

$h_1(n)$ no. of misplaced tiles

$h_2(n)$ total Manhattan distance

i.e. no. of squares from desired location of each tile

$$h_1(S) = 6$$

$$h_2(S) = 4 + 0 + 3 + 3 + 1 + 0 + 2 + 1 = 14$$

Manhattan distance is also an admissible heuristic which takes the distance from one state to another by taking a path similar to walking through city blocks.

A* Search Evaluation

A* search will follow the same path as A search when they use the same heuristic; the difference is *whether* the heuristic is admissible.

A* is **optimal** and will find the cheapest path because the $h(n)$ is admissible.

It is also **complete** because it will find the goal state if one exists.

Time complexity is exponential with path length and depends on the heuristic.

Space complexity is high because all nodes are stored.

Thus, computationally expensive but effective.

38

A* search is optimal and complete because it applies an admissible heuristic, but accomplishes this with high computational cost.

Performance of Search Strategies

Performance Measure	Breadth-First	Depth-First	Greedy Best-First	A/A*
Completeness	YES*	NO	NO	YES
Time	b^d	b^m	b^m	<i>depends on $h(n)$; exponential with path length</i>
Space	b^d	bm	b^m	b^d
Optimality	YES*	NO	NO	YES**

*BFS finds the best solution when the branching factor is finite.

**A search is optimal when the heuristic is admissible (thus being A* search)

b the maximum branching factor of the search tree

d the depth of the least-cost solution

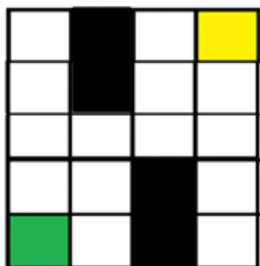
m the maximum depth of the state space problem (may be ∞)

As compared to the algorithms we have looked into thus far, A and A* search are the most effective, but certainly not the most efficient.

Practice Problem

Apply each of the following algorithms to the following problem by creating a search tree. Think of different heuristics or way to measure distance.

Problem find shortest path from tile (1,1) to (4,5) with actions up, down, left, or right.



Algorithms

BFS

DFS

DFIS

Greedy BFIS

Hill Climbing

A/A*

Use this problem to practice applying your knowledge of the search algorithms which we have covered so far. Generate a search tree that can be searched to reach tile (4,5) from (1,1). For the informed search algorithms that require using a heuristic, try applying common heuristics we have covered, such as straight-line-distance and Manhattan distance.

Intelligent Systems Framework

Efficient, simple, correct, well- understood, generic	Find best decision based on experience.	Weeks 6 & 7
	Find best decision with knowledge & reason.	Weeks 3, 4, 5
	Find best decision in multiple agent setting.	Weeks 1 & 2
	Find best decision with heuristics.	
	Systematically find all possible solutions.	

Progress in the intelligent systems framework.

Summary & Homework

Take Home Message	Homework
Informed search makes intelligent choices for node expansion based on heuristics. (Greedy) best-first search chooses the best node of its immediate neighbors. Types include hill-climbing and local beam search. A/A* search looks at the cost so far and the cost to the goal.	Practical assignment 3: Informed Search → book slots with TAs Theoretical material in work groups Practice Schnapsen

Thank you for your attention and have a lovely day.